DESIGN AND ANALYSIS OF ALGORITHMS

LAB WORKBOOK

WEEK - 5

NAME : DARUNKUMAR J

ROLL NUMBER : CH.SC.U4CSE24111

CLASS : CSE-B

Question 1: Construct an AVL tree with these numbers:

157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

(i) Print the tree showing each level.

(ii) Check that all nodes are balanced.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
typedef struct Node {
    int key;
    struct Node *left, *right;
    int height;
} Node;

// Utility function
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Get height
int height(Node *node) {
    if (node == NULL)
        return 0;
    return node->height;
}

// Create new node
Node* newNode(int key) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}

// Right rotation
Node* rightRotate(Node *y) {
    Node *x = y->left;
    Node *T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
```

```
}

// Left rotation
Node* leftRotate(Node *x) {
    Node *y = x->right;
    Node *T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

// Get balance factor
int getBalance(Node *node) {
    if (node == NULL)
        return 0;
    return height(node->left) - height(node->right);
}

// Insert into AVL tree
Node* insert(Node* node, int key) {

    if (node == NULL)
        return newNode(key);

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));

    int balance = getBalance(node);

    // LL Case
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);

    // RR Case
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);

    // LR Case
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
```

```
    // RL Case
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

// Print level order
void printLevelOrder(Node* root) {
    if (root == NULL)
        return;

    Node* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        int size = rear - front;

        while (size--) {
            Node* temp = queue[front++];
            printf("%d ", temp->key);

            if (temp->left)
                queue[rear++] = temp->left;
            if (temp->right)
                queue[rear++] = temp->right;
        }
        printf("\n"); // new level
    }
}

// Check balance for all nodes
int isBalanced(Node* root) {
    if (root == NULL)
        return 1;

    int balance = getBalance(root);

    if (balance > 1 || balance < -1)
        return 0;

    return isBalanced(root->left) && isBalanced(root->right);
}

// Main
int main() {
    Node* root = NULL;
```

```c
    int arr[] = {157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117};
    int n = sizeof(arr)/sizeof(arr[0]);

    for (int i = 0; i < n; i++) {
        root = insert(root, arr[i]);
    }

    printf("Level Order Traversal:\n");
    printLevelOrder(root);

    if (isBalanced(root))
        printf("\nAll nodes are balanced.\n");
    else
        printf("\nTree is NOT balanced.\n");

    return 0;
}
```

OUTPUT:

```
PS C:\Sem-4\DAA\week-5> gcc avltree.c
PS C:\Sem-4\DAA\week-5> ./a
Level Order Traversal:
122
111 147
110 112 133 151
117 123 141 149 157

All nodes are balanced.
```

Time Complexity

The AVL tree is a self-balancing binary search tree in which the height difference between left and right subtrees is at most one. Due to this strict balancing:

- The height of the tree is **O(log n)**.
- **Insertion of a single element** takes **O(log n)** time, as rebalancing may require traversing from the inserted node up to the root.
- **Insertion of n elements** therefore takes **O(n log n)**.
- **Rotations (LL, RR, LR, RL)** are performed in constant time, i.e., **O(1)**.
- **Level-order traversal** requires visiting all nodes, resulting in **O(n)** time complexity.
- **Balance checking** also requires visiting each node once, hence **O(n)**.

Space Complexity

- The space required to store the AVL tree with *n* nodes is **O(n)**.
- The recursion stack used during insertion has a maximum depth equal to the tree height, which is **O(log n)**.
- The auxiliary space for level-order traversal (queue) in the worst case is **O(n)**.

Question 2: Construct a Red-Black tree with the numbers:

157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117

Print the tree showing R (red) or B (black) for each node. Check that:

(i) Root is black

(ii) No red node has red children

CODE:

```
#include <stdio.h>
#include <stdlib.h>

#define RED 1
#define BLACK 0

typedef struct Node {
    int data;
    int color;
    struct Node *left, *right, *parent;
} Node;

Node *root = NULL;

// Create new node
Node* newNode(int data) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->data = data;
    node->color = RED;
    node->left = node->right = node->parent = NULL;
    return node;
}

// Left rotate
void leftRotate(Node *x) {
    Node *y = x->right;
    x->right = y->left;

    if (y->left != NULL)
        y->left->parent = x;

    y->parent = x->parent;

    if (x->parent == NULL)
        root = y;
    else if (x == x->parent->left)
        x->parent->left = y;
    else
        x->parent->right = y;
```

```
    y->left = x;
    x->parent = y;
}

// Right rotate
void rightRotate(Node *y) {
    Node *x = y->left;
    y->left = x->right;

    if (x->right != NULL)
        x->right->parent = y;

    x->parent = y->parent;

    if (y->parent == NULL)
        root = x;
    else if (y == y->parent->left)
        y->parent->left = x;
    else
        y->parent->right = x;

    x->right = y;
    y->parent = x;
}

// Fix violations
void fixInsert(Node *z) {
    while (z->parent != NULL && z->parent->color == RED) {
        Node *grandparent = z->parent->parent;

        if (z->parent == grandparent->left) {
            Node *uncle = grandparent->right;

            if (uncle != NULL && uncle->color == RED) {
                z->parent->color = BLACK;
                uncle->color = BLACK;
                grandparent->color = RED;
                z = grandparent;
            } else {
                if (z == z->parent->right) {
                    z = z->parent;
                    leftRotate(z);
                }
                z->parent->color = BLACK;
                grandparent->color = RED;
                rightRotate(grandparent);
            }
        } else {
            Node *uncle = grandparent->left;

            if (uncle != NULL && uncle->color == RED) {
                z->parent->color = BLACK;
                uncle->color = BLACK;
```

```
                grandparent->color = RED;
                z = grandparent;
            } else {
                if (z == z->parent->left) {
                    z = z->parent;
                    rightRotate(z);
                }
                z->parent->color = BLACK;
                grandparent->color = RED;
                leftRotate(grandparent);
            }
        }
    }
    root->color = BLACK;
}

// Insert
void insert(int data) {
    Node *z = newNode(data);
    Node *y = NULL;
    Node *x = root;

    while (x != NULL) {
        y = x;
        if (z->data < x->data)
            x = x->left;
        else
            x = x->right;
    }

    z->parent = y;

    if (y == NULL)
        root = z;
    else if (z->data < y->data)
        y->left = z;
    else
        y->right = z;

    fixInsert(z);
}

// Level order print
void printLevelOrder() {
    if (root == NULL) return;

    Node* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        int size = rear - front;
```

```c
        while (size--) {
            Node* temp = queue[front++];
            printf("%d(%c) ", temp->data,
                   temp->color == RED ? 'R' : 'B');

            if (temp->left)
                queue[rear++] = temp->left;
            if (temp->right)
                queue[rear++] = temp->right;
        }
        printf("\n");
    }
}

// Check root black
int isRootBlack() {
    return (root != NULL && root->color == BLACK);
}

// Check no red-red violation
int checkRedViolation(Node *node) {
    if (node == NULL) return 1;

    if (node->color == RED) {
        if ((node->left && node->left->color == RED) ||
            (node->right && node->right->color == RED))
            return 0;
    }

    return checkRedViolation(node->left) &&
           checkRedViolation(node->right);
}

// Main
int main() {
    int arr[] = {157, 110, 147, 122, 149, 151, 111, 141, 112, 123, 133, 117};
    int n = sizeof(arr)/sizeof(arr[0]);

    for (int i = 0; i < n; i++)
        insert(arr[i]);

    printf("Level Order Traversal (with colors):\n");
    printLevelOrder();

    printf("\nRoot is %s\n", isRootBlack() ? "BLACK ✓" : "NOT BLACK ✗");

    printf("No Red-Red violation: %s\n",
           checkRedViolation(root) ? "YES ✓" : "NO ✗");

    return 0;
}
```

OUTPUT:

```
PS C:\Sem-4\DAA\week-5> gcc redblack.c
PS C:\Sem-4\DAA\week-5> ./a
Level Order Traversal (with colors):
122(B)
111(R) 147(R)
110(B) 112(B) 133(B) 151(B)
117(R) 123(R) 141(R) 149(R) 157(R)

Root is BLACK Γ£ö
No Red-Red violation: YES Γ£ö
```

Time Complexity

A Red-Black tree is a self-balancing binary search tree with less strict balancing compared to AVL trees. Its height is always bounded by **O(log n)**.

- **Insertion of a single element** takes **O(log n)** time.
- **Insertion of n elements** takes **O(n log n)**.
- **Rotations and recoloring operations** during fixing of violations take **O(1)** time per insertion.
- **Level-order traversal** takes **O(n)** time as all nodes are visited.
- **Checking for red-red violations** requires traversal of all nodes, hence **O(n)**.

Space Complexity

- The space required to store *n* nodes is **O(n)**.
- The recursion depth (for checking violations) is bounded by the tree height, i.e., **O(log n)**.
- The queue used for level-order traversal requires **O(n)** space in the worst case.